



Red Hat OpenShift AI Self-Managed 3.0

Deploying models

Deploy models in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.0 Deploying models

Deploy models in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © 2025 Red Hat, Inc.

The text of and illustrations in this document are licensed by Red Hat under a Creative Commons Attribution–Share Alike 3.0 Unported license ("CC-BY-SA"). An explanation of CC-BY-SA is available at

<http://creativecommons.org/licenses/by-sa/3.0/>

. In accordance with CC-BY-SA, if you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, Red Hat Enterprise Linux, the Shadowman logo, the Red Hat logo, JBoss, OpenShift, Fedora, the Infinity logo, and RHCE are trademarks of Red Hat, Inc., registered in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

Java[®] is a registered trademark of Oracle and/or its affiliates.

XFS[®] is a trademark of Silicon Graphics International Corp. or its subsidiaries in the United States and/or other countries.

MySQL[®] is a registered trademark of MySQL AB in the United States, the European Union and other countries.

Node.js[®] is an official trademark of Joyent. Red Hat is not formally related to or endorsed by the official Joyent Node.js open source or commercial project.

The OpenStack[®] Word Mark and OpenStack logo are either registered trademarks/service marks or trademarks/service marks of the OpenStack Foundation, in the United States and other countries and are used with the OpenStack Foundation's permission. We are not affiliated with, endorsed or sponsored by the OpenStack Foundation, or the OpenStack community.

All other trademarks are the property of their respective owners.

Abstract

As a Red Hat OpenShift AI user, you can deploy your machine-learning models in Red Hat OpenShift AI Self-Managed.

Table of Contents

CHAPTER 1. STORING MODELS	3
1.1. USING OCI CONTAINERS FOR MODEL STORAGE	3
1.2. STORING A MODEL IN AN OCI IMAGE	3
1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)	5
CHAPTER 2. DEPLOYING MODELS	7
2.1. DEPLOYING MODELS ON THE MODEL SERVING PLATFORM	7
2.2. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI	10
2.3. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D	12
2.3.1. Configuring authentication for Distributed Inference with llm-d using Red Hat Connectivity Link	13
2.3.2. Enabling Distributed Inference with llm-d	14
2.3.3. Example usage for Distributed Inference with llm-d	16
2.3.3.1. Single-node GPU deployment	16
2.3.3.2. Multi-node deployment	16
2.3.3.3. Intelligent inference scheduler with KV cache routing	16
2.4. MONITORING MODELS	16
2.4.1. Viewing performance metrics for a deployed model	16
2.4.2. Viewing model-serving runtime metrics for the model serving platform	18
CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM	20
3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM	20
3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL	22
3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL	23
CHAPTER 4. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS	25
4.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL	25
4.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL	25
4.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE MODEL SERVING PLATFORM	26
4.4. INFERENCE ENDPOINTS	26
4.4.1. Caikit TGIS ServingRuntime for KServe	26
4.4.2. OpenVINO Model Server	26
4.4.3. vLLM NVIDIA GPU ServingRuntime for KServe	27
4.4.4. vLLM Intel Gaudi Accelerator ServingRuntime for KServe	28
4.4.5. vLLM AMD GPU ServingRuntime for KServe	28
4.4.6. vLLM Spyre AI Accelerator ServingRuntime for KServe	28
4.4.7. vLLM Spyre s390x ServingRuntime for KServe	28
4.4.8. NVIDIA Triton Inference Server	28
4.4.9. Seldon MLServer	29
4.4.10. Additional resources	30

CHAPTER 1. STORING MODELS

You must store your model before you can deploy it. You can store a model in an S3 bucket, URI or Open Container Initiative (OCI) containers.

1.1. USING OCI CONTAINERS FOR MODEL STORAGE

As an alternative to storing a model in an S3 bucket or URI, you can upload models to Open Container Initiative (OCI) containers. Deploying models from OCI containers is also known as modelcars in KServe.

Using OCI containers for model storage can help you:

- Reduce startup times by avoiding downloading the same model multiple times.
- Reduce disk space usage by reducing the number of models downloaded locally.
- Improve model performance by allowing pre-fetched images.

Using OCI containers for model storage involves the following tasks:

- Storing a model in an OCI image.
- Deploying a model from an OCI image by using either the user interface or the command line interface. To deploy a model by using:
 - The user interface, see [Deploying models on the model serving platform](#).
 - The command line interface, see [Deploying a model stored in an OCI image by using the CLI](#).

1.2. STORING A MODEL IN AN OCI IMAGE

You can store a model in an OCI image. The following procedure uses the example of storing a MobileNet v2-7 model in ONNX format.

Prerequisites

- You have a model in the ONNX format. The example in this procedure uses the MobileNet v2-7 model in ONNX format.
- You have installed the Podman tool.

Procedure

1. In a terminal window on your local machine, create a temporary directory for storing both the model and the support files that you need to create the OCI image:

```
cd $(mktemp -d)
```

2. Create a **models** folder inside the temporary directory:

```
mkdir -p models/1
```

**NOTE**

This example command specifies the subdirectory **1** because OpenVINO requires numbered subdirectories for model versioning. If you are not using OpenVINO, you do not need to create the **1** subdirectory to use OCI container images.

- Download the model and support files:

```
DOWNLOAD_URL=https://github.com/onnx/models/raw/main/validated/vision/classification/mobilenet/model/mobilenetv2-7.onnx
curl -L $DOWNLOAD_URL -O --output-dir models/1/
```

- Use the **tree** command to confirm that the model files are located in the directory structure as expected:

```
tree
```

The **tree** command should return a directory structure similar to the following example:

```
.
├── Containerfile
├── models
│   └── 1
│       └── mobilenetv2-7.onnx
```

- Create a Docker file named **Containerfile**:

**NOTE**

- Specify a base image that provides a shell. In the following example, **ubi9-micro** is the base container image. You cannot specify an empty image that does not provide a shell, such as **scratch**, because KServe uses the shell to ensure the model files are accessible to the model server.
- Change the ownership of the copied model files and grant read permissions to the root group to ensure that the model server can access the files. OpenShift runs containers with a random user ID and the root group ID.

```
FROM registry.access.redhat.com/ubi9/ubi-micro:latest
COPY --chown=0:0 models /models
RUN chmod -R a=rX /models

# nobody user
USER 65534
```

- Use **podman build** commands to create the OCI container image and upload it to a registry. The following commands use Quay as the registry.

**NOTE**

If your repository is private, ensure that you are authenticated to the registry before uploading your container image.


```
podman build --format=oci -t quay.io/<user_name>/<repository_name>:<tag_name> .
podman push quay.io/<user_name>/<repository_name>:<tag_name>
```

1.3. UPLOADING MODEL FILES TO A PERSISTENT VOLUME CLAIM (PVC)

When deploying a model, you can serve it from a preexisting Persistent Volume Claim (PVC) where your model files are stored. You can upload your local model files to a PVC in the IDE that you access from a running workbench.

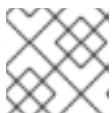
Prerequisites

- You have access to the OpenShift AI dashboard.
- You have access to a project that has a running workbench.
- You have created a persistent volume claim (PVC) with a context type of **Model storage**.
- The workbench is attached to the persistent volume (PVC).
 - For instructions on attaching a PVC, see [Creating a project workbench](#).
- You have the model files saved on your local machine.

Procedure

Follow these steps to upload your model files to the PVC mount point (**/opt/app-root/src/**) within your workbench:

1. From the OpenShift AI dashboard, click the open icon () to open your IDE in a new window.
2. In your IDE, navigate to the **File Browser** pane on the left-hand side.
 - a. In JupyterLab, this is usually labeled **Files**.
 - b. In code-server, this is usually the **Explorer** view.
3. In the file browser, navigate to the **/opt/app-root/src/** folder. This folder represents the root of your attached PVC.



NOTE

Any files or folders that you create or upload to this folder persist in the PVC.

4. Optional: Create a new folder to organize your models:
 - a. In the file browser, right-click within the **/opt/app-root/src/** folder in the file browser and select **New Folder**.
 - b. Name the folder (for example, **models**).
 - c. Double-click the new **models** folder to enter it.
5. Upload your model files to the current folder (**/opt/app-root/src/** or **/opt/app-root/src/models/**):

- **Using JupyterLab:**

- a. Click the **Upload Files** icon () in the file browser toolbar above the folder listing.
- b. In the file selection dialog, navigate to and select the model files from your local computer. Click **Open**.
- c. Wait for the upload progress bars next to the filenames to complete.

- **Using code-server:**

- a. Drag the model files directly from your local file explorer and drop them into the file browser pane in the target folder within code-server.
6. Wait for the upload process to complete.

Verification

- Confirm that your files appear in the file browser at the path where you uploaded them.

Next steps

When you follow the procedure to deploy a model, you can access the model files from the specified path within your PVC:

1. In the **Deploy model** dialog, select **Existing cluster storage** under the **Source model location** section.
2. From the **Cluster storage** list, select the PVC associated with your workbench.
3. In the **Model path** field, enter the path to your model or the folder containing your model.

CHAPTER 2. DEPLOYING MODELS

The model serving platform is based on the KServe component and deploys each model from its own dedicated model server. This architecture is ideal for deploying, monitoring, scaling, and maintaining large models that require more resources, such as large language models (LLMs).

2.1. DEPLOYING MODELS ON THE MODEL SERVING PLATFORM

You can deploy Generative AI (GenAI) or Predictive AI models on the model serving platform by using the **Deploy a model** wizard. The wizard allows you to configure your model, including specifying its location and type, selecting a serving runtime, assigning a hardware profile, and setting advanced configurations like external routes and token authentication.

To successfully deploy a model, you must meet the following prerequisites.

General prerequisites

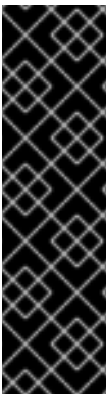
- You have logged in to Red Hat OpenShift AI.
- You have installed KServe and enabled the model serving platform.
- You have enabled a preinstalled or custom model-serving runtime.
- You have created a project.
- You have access to S3-compatible object storage, a URI-based repository, an OCI-compliant registry or a persistent volume claim (PVC) and have added a connection to your project. For more information about adding a connection, see [Adding a connection to your project](#).
- If you want to use graphics processing units (GPUs) with your model server, you have enabled GPU support in OpenShift AI. If you use NVIDIA GPUs, see [Enabling NVIDIA GPUs](#). If you use AMD GPUs, see [AMD GPU integration](#).

Runtime-specific prerequisites

Meet the requirements for the specific runtime you intend to use.

- **Caikit-TGIS runtime**
 - To use the Caikit-TGIS runtime, you have converted your model to Caikit format. For an example, see [Converting Hugging Face Hub models to Caikit format](#) in the [caikit-tgis-serving](#) repository.
- **vLLM NVIDIA GPU ServingRuntime for KServe**
 - To use the **vLLM NVIDIA GPU ServingRuntime for KServe** runtime, you have enabled GPU support in OpenShift AI and have installed and configured the Node Feature Discovery Operator on your cluster. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- **vLLM CPU ServingRuntime for KServe**
 - To use the VLLM runtime on IBM Z and IBM Power, use the **vLLM CPU ServingRuntime for KServe**. You cannot use GPU accelerators with IBM Z and IBM Power architectures. For more information, see [Red Hat OpenShift Multi Architecture Component Availability Matrix](#).
- **vLLM Intel Gaudi Accelerator ServingRuntime for KServe**

- To use the **vLLM Intel Gaudi Accelerator ServingRuntime for KServer**runtime, you have enabled support for hybrid processing units (HPUs) in OpenShift AI. This includes installing the Intel Gaudi Base Operator and configuring a hardware profile. For more information, see [Intel Gaudi Base Operator OpenShift installation](#) in the AMD documentation and [Working with hardware profiles](#).
- **vLLM AMD GPU ServingRuntime for KServe**
 - To use the **vLLM AMD GPU ServingRuntime for KServer**runtime, you have enabled support for AMD graphic processing units (GPUs) in OpenShift AI. This includes installing the AMD GPU operator and configuring a hardware profile. For more information, see [Deploying the AMD GPU operator on OpenShift](#) and [Working with hardware profiles](#).
- **vLLM Spyre AI Accelerator ServingRuntime for KServe**



IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

- To use the **vLLM Spyre AI Accelerator ServingRuntime for KServer**runtime on x86, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).
 - **vLLM Spyre s390x ServingRuntime for KServe**
- To use the **vLLM Spyre s390x ServingRuntime for KServer**runtime on IBM Z, you have installed the Spyre Operator and configured a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

Procedure

1. In the left menu, click **Projects**.
2. Click the name of the project that you want to deploy a model in.
A project details page opens.
3. Click the **Deployments** tab.
4. Click the **Deploy model** button.
The **Deploy a model** wizard opens.
5. In the **Model details** section, provide information about the model:
 - a. From the **Model location** list, specify where your model is stored and complete the connection detail fields.

**NOTE**

- The **OCI-compliant registry**, **S3 compatible object storage**, and **URI** options are pre-installed connection types. Additional options might be available if your OpenShift AI administrator added them.
- If you have uploaded model files to a persistent volume claim (PVC) and the PVC is attached to your workbench, the **Cluster storage** option becomes available in the **Model location** list. Use this option to select the PVC and specify the path to the model file.

- b. From the **Model type** list, select the type of model that you are deploying, **Predictive** or **Generative AI model**.
 - c. Click **Next**.
6. In the **Model deployment** section, configure the deployment:
 - a. In the **Model deployment name** field, enter a unique name for your model deployment.
 - b. In the **Description** field, enter a description of your deployment.
 - c. From the **Hardware profile** list, select a hardware profile.
 - d. Optional: To modify the default resource allocation, click **Customize resource requests and limits** and enter new values for the CPU and Memory requests and limits.
 - e. In the **Serving runtime** field, select an enabled runtime.

**NOTE**

If project-scoped runtimes exist, the **Serving runtime** list includes subheadings to distinguish between global runtimes and project-scoped runtimes.

- f. Optional: If you selected a **Predictive model** type, select a framework from the **Model framework (name - version)** list. This field is hidden for Generative AI models.
 - g. In the **Number of model server replicas to deploy** field, specify a value.
 - h. Click **Next**.
7. In the **Advanced settings** section, configure advanced options:
 - a. Optional: (Generative AI models only) Select the **Add as AI asset endpoint** checkbox if you want to add your model's endpoint to the **AI asset endpoints** page.
 - i. In the **Use case** field, enter the types of tasks that your model performs, such as chat, multimodal, or natural language processing.

**NOTE**

You must add your model as an AI asset endpoint to test your model in the GenAI playground.

- b. Optional: Select the **Model access** checkbox to make your model deployment available through an external route.
 - c. Optional: To require token authentication for inference requests to the deployed model, select **Require token authentication**.
 - d. In the **Service account name** field, enter the service account name that the token will be generated for.
 - e. To add an additional service account, click **Add a service account** and enter another service account name.
 - f. Optional: In the **Configuration parameters** section:
 - i. Select the **Add custom runtime arguments** and then enter arguments in the text field.
 - ii. Select the **Add custom runtime environment variables** checkbox, then click **Add variable** to enter custom variables in the text field.
8. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.

Additional resources

- [Model-serving runtimes for accelerators](#)
- [Adding a custom model-serving runtime](#).

2.2. DEPLOYING A MODEL STORED IN AN OCI IMAGE BY USING THE CLI

You can deploy a model that is stored in an OCI image from the command line interface.

The following procedure uses the example of deploying a MobileNet v2-7 model in ONNX format, stored in an OCI image on an OpenVINO model server.



NOTE

By default in KServe, models are exposed outside the cluster and not protected with authentication.

Prerequisites

- You have stored a model in an OCI image as described in [Storing a model in an OCI image](#).
- If you want to deploy a model that is stored in a private OCI repository, you must configure an image pull secret. For more information about creating an image pull secret, see [Using image pull secrets](#).
- You are logged in to your OpenShift cluster.

Procedure

Procedure

1. Create a project to deploy the model:

```
oc new-project oci-model-example
```

2. Use the OpenShift AI Applications project **kserve-ovms** template to create a **ServingRuntime** resource and configure the OpenVINO model server in the new project:

```
oc process -n redhat-ods-applications -o yaml kserve-ovms | oc apply -f -
```

3. Verify that the **ServingRuntime** named **kserve-ovms** is created:

```
oc get servingruntimes
```

The command should return output similar to the following:

NAME	DISABLED	MODELTYPE	CONTAINERS	AGE
kserve-ovms		openvino_ir	kserve-container	1m

4. Create an **InferenceService** YAML resource, depending on whether the model is stored from a private or a public OCI repository:

- For a model stored in a public OCI repository, create an **InferenceService** YAML file with the following values, replacing **<user_name>**, **<repository_name>**, and **<tag_name>** with values specific to your environment:

```
apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-oci
spec:
  predictor:
    model:
      runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
```

- For a model stored in a private OCI repository, create an **InferenceService** YAML file that specifies your pull secret in the **spec.predictor.imagePullSecrets** field, as shown in the following example:

—

```

apiVersion: serving.kserve.io/v1beta1
kind: InferenceService
metadata:
  name: sample-isvc-using-private-oci
spec:
  predictor:
    model:
      runtime: kserve-ovms # Ensure this matches the name of the ServingRuntime
resource
  modelFormat:
    name: onnx
  storageUri: oci://quay.io/<user_name>/<repository_name>:<tag_name>
  resources:
    requests:
      memory: 500Mi
      cpu: 100m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    limits:
      memory: 4Gi
      cpu: 500m
      # nvidia.com/gpu: "1" # Only required if you have GPUs available and the model
and runtime will use it
    imagePullSecrets: # Specify image pull secrets to use for fetching container images,
including OCI model images
      - name: <pull-secret-name>

```

After you create the **InferenceService** resource, KServe deploys the model stored in the OCI image referred to by the **storageUri** field.

Verification

Check the status of the deployment:

```
oc get inferencservice
```

The command should return output that includes information, such as the URL of the deployed model and its readiness state.

2.3. DEPLOYING MODELS BY USING DISTRIBUTED INFERENCE WITH LLM-D

Distributed Inference with llm-d is a Kubernetes-native, open-source framework designed for serving large language models (LLMs) at scale. You can use Distributed Inference with llm-d to simplify the deployment of generative AI, focusing on high performance and cost-effectiveness across various hardware accelerators.

Key features of Distributed Inference with llm-d include:

- Efficiently handles large models using optimizations such as prefix-cache aware routing and disaggregated serving.
- Integrates into a standard Kubernetes environment, where it leverages specialized components like the Envoy proxy to handle networking and routing, and high-performance libraries such as vLLM and NVIDIA Inference Transfer Library (NIXL).

- Tested recipes and well-known presets reduce the complexity of deploying inference at scale, so users can focus on building applications rather than managing infrastructure.

Serving models using Distributed Inference with llm-d on Red Hat OpenShift AI consists of the following steps:

1. Installing OpenShift AI.
2. Enabling the model serving platform.
3. Configuring authentication with Red Hat Connectivity Link.
4. Enabling Distributed Inference with llm-d on a Kubernetes cluster.
5. Creating an LLMInferenceService Custom Resource (CR).
6. Deploying a model.

2.3.1. Configuring authentication for Distributed Inference with llm-d using Red Hat Connectivity Link

Red Hat Connectivity Link is used for authentication and authorization.

Prerequisites

- You have installed Red Hat Connectivity Link version 1.1.1 or later. For more information, see [Installing Connectivity Link on OpenShift](#).
- You have access to the OpenShift CLI (**oc**).
- The ServiceAccount has permission to get the corresponding **LLMInferenceService** and you have generated a JSON web token (JWT).

Procedure

1. Create the Kuadrant custom resource (CR) to set up required objects:

```
oc apply -f - <<EOF
apiVersion: kuadrant.io/v1beta1
kind: Kuadrant
metadata:
  name: kuadrant
  namespace: kuadrant-system
EOF
```

2. Wait for Kuadrant to become ready:

```
oc wait Kuadrant -n kuadrant-system kuadrant --for=condition=Ready --timeout=10m
```

3. Add the **ServingCert** annotation to the Authorino Service:

```
oc annotate svc/authorino-authorino-authorization service.beta.openshift.io/serving-cert-secret-name=authorino-server-cert -n kuadrant-system
```

- Wait for the secret to be created:

```
sleep 2
```

- Update Authorino to enable SSL:

```
oc apply -f - <<EOF
apiVersion: operator.authorino.kuadrant.io/v1beta1
kind: Authorino
metadata:
  name: authorino
  namespace: kuadrant-system
spec:
  replicas: 1
  clusterWide: true
  listener:
    tls:
      enabled: true
      certSecretRef:
        name: authorino-server-cert
  oidcServer:
    tls:
      enabled: false
EOF
```

- Verify that the Authorino pods are ready:

```
oc wait --for=condition=ready pod -l authorino-resource=authorino -n kuadrant-system --
timeout 150s
```

- If OpenShift AI was installed before installing Connectivity Link and Kuadrant, restart the controllers:

```
oc delete pod -n redhat-ods-applications -l app=odh-model-controller
oc delete pod -n redhat-ods-applications -l control-plane=kserve-controller-manager
```

2.3.2. Enabling Distributed Inference with llm-d

This procedure describes how to create a custom resource (CR) for an **LLMInferenceService** resource. You replace the default **InferenceService** with the **LLMInferenceService**.

Prerequisites

- You have enabled the model serving platform.
- You have access to an OpenShift cluster running version 4.19.9 or later.
- OpenShift Service Mesh v2 is not installed in the cluster.
- Your cluster administrator has created a **GatewayClass** and a **Gateway** named **openshift-ai-inference** in the **openshift-ingress** namespace as described in [Gateway API with OpenShift Container Platform Networking](#).



IMPORTANT

Review the [Gateway API deployment topologies](#). Only use shared Gateways across trusted namespaces.

- Your cluster administrator has installed the **LeaderWorkerSet** Operator in OpenShift. For more information, see the [Leader Worker Set Operator documentation](#).
- If you are running OpenShift on a bare metal cluster: Your cluster administrator has set up the MetalLB Operator to provision an external IP address for the **openshift-ai-inference** Gateway service with the type **LoadBalancer**. For more information, see [Load balancing with MetalLB](#). Ensure that the LoadBalancer is configured as follows:
 - Has a standard Kubernetes Service manifest.
 - Has **type:LoadBalancer** in the **spec** section.
- You have enabled authentication as described in *Configuring authentication for Distributed Inference with llm-d*.

Procedure

1. Log in to the OpenShift console as a developer.
2. Create the **LLMInferenceService** CR with the following information:

```
apiVersion: serving.kserve.io/v1alpha1
kind: LLMInferenceService
metadata:
  name: sample-llm-inference-service
spec:
  replicas: 2
  model:
    uri: hf://RedHatAI/Qwen3-8B-FP8-dynamic
    name: RedHatAI/Qwen3-8B-FP8-dynamic
  router:
    route: {}
    gateway: {}
    scheduler: {}
  template:
    containers:
      - name: main
        resources:
          limits:
            cpu: '4'
            memory: 32Gi
            nvidia.com/gpu: "1"
          requests:
            cpu: '2'
            memory: 16Gi
            nvidia.com/gpu: "1"
```

Customize the following parameters in the **spec** section of the inference service:

- **replicas** - Specify the number of replicas.

- **model** - Provide the URI to the model based on how the model is stored (**uri**) and the model name to use in chat completion requests (**name**).
 - S3 bucket: **s3://<bucket-name>/<object-key>**
 - Persistent volume claim (PVC): **pvc://<claim-name>/<pvc-path>**
 - OCI container image: **oci://<registry_host>/<org_or_username>/<repository_name><tag_or_digest>**
 - HuggingFace: **hf://<model>/<optional-hash>**
- **router** - Provide an HTTPRoute and gateway, or leave blank to automatically create one.

3. Save the file.

2.3.3. Example usage for Distributed Inference with llm-d

These examples show how to use Distributed Inference with llm-d in common scenarios.

2.3.3.1. Single-node GPU deployment

Use single-GPU-per-replica deployment patterns for development, testing, or production deployments of smaller models, such as 7-billion-parameter models.

For examples using single-node GPU deployments, see [Single-Node GPU Deployment Examples](#).

2.3.3.2. Multi-node deployment

For examples using multi-node deployments, see [DeepSeek-R1 Multi-Node Deployment Examples](#).

2.3.3.3. Intelligent inference scheduler with KV cache routing

You can configure the scheduler to track key-value (KV) cache blocks across inference endpoints and route requests to the endpoint with the highest cache hit rate. This configuration improves throughput and reduces latency by maximizing cache reuse.

For an example, see [Precise Prefix KV Cache Routing](#).

2.4. MONITORING MODELS

You can monitor models that are deployed on the model serving platform to view performance and resource usage metrics.

2.4.1. Viewing performance metrics for a deployed model

You can monitor the following metrics for a specific model that is deployed on the model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)**- The average time it takes a specific model to respond to requests.

- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, when the peak usage hours are and how the model is performing at a specified time.

Prerequisites

- You have installed Red Hat OpenShift AI.
- A cluster admin has enabled user workload monitoring (UWM) for user-defined projects on your OpenShift cluster. For more information, see [Enabling monitoring for user-defined projects](#) and [Configuring monitoring for the model serving platform](#).
- You have logged in to Red Hat OpenShift AI.
- The following dashboard configuration options are set to the default values as shown:

```
disablePerformanceMetrics:false
disableKServeMetrics:false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

- You have deployed a model on the model serving platform by using a preinstalled runtime.



NOTE

Metrics are only supported for models deployed by using a preinstalled model-serving runtime or a custom runtime that is duplicated from a preinstalled runtime.

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.
The **Projects** page opens.
2. Click the name of the project that contains the data science models that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Select the model that you are interested in.
5. On the **Endpoint performance** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.

6. Scroll down to view data graphs for number of requests, average response time, CPU utilization, and memory utilization.

Verification

The **Endpoint performance** tab shows graphs of metrics for the model.

2.4.2. Viewing model-serving runtime metrics for the model serving platform

When a cluster administrator has configured monitoring for the model serving platform, non-admin users can use the OpenShift web console to view model-serving runtime metrics for the KServe component.

Prerequisites

- A cluster administrator has configured monitoring for the model serving platform.
- You have been assigned the **monitoring-rules-view** role. For more information, see [Granting users permission to configure monitoring for user-defined projects](#).
- You are familiar with how to monitor project metrics in the OpenShift web console. For more information, see [Monitoring your project metrics](#).

Procedure

1. Log in to the OpenShift web console.
2. Switch to the **Developer** perspective.
3. In the left menu, click **Observe**.
4. As described in [Monitoring your project metrics](#), use the web console to run queries for model-serving runtime metrics. You can also run queries for metrics that are related to OpenShift Service Mesh. Some examples are shown.
 - a. The following query displays the number of successful inference requests over a period of time for a model deployed with the vLLM runtime:

```
sum(increase(vllm:request_success_total{namespace=${namespace},model_name=${model_name}}[${rate_interval}]))
```



NOTE

Certain vLLM metrics are available only after an inference request is processed by a deployed model. To generate and view these metrics, you must first make an inference request to the model.

- b. The following query displays the number of successful inference requests over a period of time for a model deployed with the OpenVINO Model Server runtime:

```
sum(increase(ovms_requests_success{namespace=${namespace},name=${model_name}}[${rate_interval}]))
```

Additional resources

- [OVMS metrics](#)

CHAPTER 3. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

You can deploy models using NVIDIA NIM inference services on the NVIDIA NIM model serving platform.

NVIDIA NIM, part of NVIDIA AI Enterprise, is a set of microservices designed for secure, reliable deployment of high performance AI model inferencing across clouds, data centers and workstations.

3.1. DEPLOYING MODELS ON THE NVIDIA NIM MODEL SERVING PLATFORM

When you have enabled the **NVIDIA NIM model serving platform**, you can start to deploy NVIDIA-optimized models on the platform.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have enabled the **NVIDIA NIM model serving platform**
- You have created a project.
- You have enabled support for graphic processing units (GPUs) in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).

Procedure

1. In the left menu, click **Projects**.
The **Projects** page opens.
2. Click the name of the project that you want to deploy a model in.
A project details page opens.
3. Click the **Deployments** tab.
4. In the **Deployments** section, perform one of the following actions:
 - On the **NVIDIA NIM model serving platform** tile, click **Select NVIDIA NIM** on the tile, and then click **Deploy model**.
 - If you have previously selected the NVIDIA NIM model serving type, the **Deployments** page displays **NVIDIA model serving enabled** on the upper-right corner, along with the **Deploy model** button. To proceed, click **Deploy model**.

The **Deploy model** dialog opens.

5. Configure properties for deploying your model as follows:
 - a. In the **Model deployment name** field, enter a unique name for the deployment.
 - b. From the **NVIDIA NIM** list, select the NVIDIA NIM model that you want to deploy. For more information, see [Supported Models](#)

- c. In the **NVIDIA NIM storage size** field, specify the size of the cluster storage instance that will be created to store the NVIDIA NIM model.



NOTE

When resizing a PersistentVolumeClaim (PVC) backed by Amazon EBS in OpenShift AI, you may encounter **VolumeModificationRateExceeded: You've reached the maximum modification rate per volume limit**. To avoid this error, wait at least six hours between modifications per EBS volume. If you resize a PVC before the cooldown expires, the Amazon EBS CSI driver (ebs.csi.aws.com) fails with this error. This error is an Amazon EBS service limit that applies to all workloads using EBS-backed PVCs.

- d. In the **Number of model server replicas to deploy** field, specify a value.
 - e. From the **Model server size** list, select a value.
6. From the **Hardware profile** list, select a hardware profile.
 7. Optional: Click **Customize resource requests and limit** and update the following values:
 - a. In the **CPUs requests** field, specify the number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
 - b. In the **CPU limits** field, specify the maximum number of CPUs to use with your model server. Use the list beside this field to specify the value in cores or millicores.
 - c. In the **Memory requests** field, specify the requested memory for the model server in gibibytes (Gi).
 - d. In the **Memory limits** field, specify the maximum memory limit for the model server in gibibytes (Gi).
 8. Optional: In the **Model route** section, select the **Make deployed models available through an external route** checkbox to make your deployed models available to external clients.
 9. To require token authentication for inference requests to the deployed model, perform the following actions:
 - a. Select **Require token authentication**.
 - b. In the **Service account name** field, enter the service account name that the token will be generated for.
 - c. To add an additional service account, click **Add a service account** and enter another service account name.
 10. Click **Deploy**.

Verification

- Confirm that the deployed model is shown on the **Deployments** tab for the project, and on the **Deployments** page of the dashboard with a checkmark in the **Status** column.

Additional resources

- [NVIDIA NIM API reference](#)
- [Supported Models](#)

3.2. VIEWING NVIDIA NIM METRICS FOR A NIM MODEL

In OpenShift AI, you can observe the following NVIDIA NIM metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- GPU cache usage over time (ms)
- Current running, waiting, and max requests count
- Tokens count
- Time to first token
- Time per output token
- Request outcomes

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.
- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.
The **Projects** page opens.
2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Click the NIM model that you want to observe.
5. On the **NIM Metrics** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.

- **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed (to show the latest data). You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.

6. Scroll down to view data graphs for NIM metrics.

Verification

The **NIM Metrics** tab shows graphs of NIM metrics for the deployed NIM model.

Additional resources

- [NVIDIA NIM observability](#)

3.3. VIEWING PERFORMANCE METRICS FOR A NIM MODEL

You can observe the following performance metrics for a NIM model deployed on the NVIDIA NIM model serving platform:

- **Number of requests** - The number of requests that have failed or succeeded for a specific model.
- **Average response time (ms)** - The average time it takes a specific model to respond to requests.
- **CPU utilization (%)** - The percentage of the CPU limit per model replica that is currently utilized by a specific model.
- **Memory utilization (%)** - The percentage of the memory limit per model replica that is utilized by a specific model.

You can specify a time range and a refresh interval for these metrics to help you determine, for example, the peak usage hours and model performance at a specified time.

Prerequisites

- You have enabled the NVIDIA NIM model serving platform.
- You have deployed a NIM model on the NVIDIA NIM model serving platform.
- A cluster administrator has enabled metrics collection and graph generation for your deployment.
- The **disableKServeMetrics** OpenShift AI dashboard configuration option is set to its default value of **false**:

```
disableKServeMetrics: false
```

For more information about setting dashboard configuration options, see [Customizing the dashboard](#).

Procedure

1. From the OpenShift AI dashboard navigation menu, click **Projects**.
The **Projects** page opens.

2. Click the name of the project that contains the NIM model that you want to monitor.
3. In the project details page, click the **Deployments** tab.
4. Click the NIM model that you want to observe.
5. On the **Endpoint performance** tab, set the following options:
 - **Time range** - Specifies how long to track the metrics. You can select one of these values: 1 hour, 24 hours, 7 days, and 30 days.
 - **Refresh interval** - Specifies how frequently the graphs on the metrics page are refreshed to show the latest data. You can select one of these values: 15 seconds, 30 seconds, 1 minute, 5 minutes, 15 minutes, 30 minutes, 1 hour, 2 hours, and 1 day.
6. Scroll down to view data graphs for performance metrics.

Verification

The **Endpoint performance** tab shows graphs of performance metrics for the deployed NIM model.

CHAPTER 4. MAKING INFERENCE REQUESTS TO DEPLOYED MODELS

When you deploy a model, it is available as a service that you can access with API requests. This allows you to get predictions from your model based on the data you provide in the request.

4.1. ACCESSING THE AUTHENTICATION TOKEN FOR A DEPLOYED MODEL

If you secured your model inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify it in your inference requests.

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the model serving platform.

Procedure

1. From the OpenShift AI dashboard, click **Projects**.
The **Projects** page opens.
2. Click the name of the project that contains your deployed model.
A project details page opens.
3. Click the **Deployments** tab.
4. In the **Deployments** list, expand the section for your model.
Your authentication token is shown in the **Token authentication** section, in the **Token secret** field.
5. Optional: To copy the authentication token for use in an inference request, click the **Copy** button () next to the token value.

4.2. ACCESSING THE INFERENCE ENDPOINT FOR A DEPLOYED MODEL

To make inference requests to your deployed model, you must know how to access the inference endpoint that is available.

For a list of paths to use with the supported runtimes and example commands, see [Inference endpoints](#).

Prerequisites

- You have logged in to Red Hat OpenShift AI.
- You have deployed a model by using the model serving platform.
- If you enabled token authentication for your deployed model, you have the associated token value.

Procedure

1. From the OpenShift AI dashboard, click **AI hub** → **Deployments**.
The inference endpoint for the model is shown in the **Inference endpoints** field.
2. Depending on what action you want to perform with the model (and if the model supports that action), copy the inference endpoint and then add a path to the end of the URL.
3. Use the endpoint to make API requests to your deployed model.

Additional resources

- [Caikit API documentation](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)

4.3. MAKING INFERENCE REQUESTS TO MODELS DEPLOYED ON THE MODEL SERVING PLATFORM

When you deploy a model by using the model serving platform, the model is available as a service that you can access using API requests. This enables you to return predictions based on data inputs. To use API requests to interact with your deployed model, you must know the inference endpoint for the model.

In addition, if you secured your inference endpoint by enabling token authentication, you must know how to access your authentication token so that you can specify this in your inference requests.

4.4. INFERENCE ENDPOINTS

These examples show how to use inference endpoints to query the model.



NOTE

If you enabled token authentication when deploying the model, add the **Authorization** header and specify a token value.

4.4.1. Caikit TGIS ServingRuntime for KServe

- **:443/api/v1/task/text-generation**
- **:443/api/v1/task/server-streaming-text-generation**

Example command

```
curl --json '{"model_id": "<model_name__>", "inputs": "<text>"}'  
https://<inference_endpoint_url>:443/api/v1/task/server-streaming-text-generation -H 'Authorization:  
Bearer <token>'
```

4.4.2. OpenVINO Model Server

- **/v2/models/<model-name>/infer**

Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data>] } ]}' -H 'Authorization: Bearer <token>'
```

4.4.3. vLLM NVIDIA GPU ServingRuntime for KServe

- **:443/version**
- **:443/docs**
- **:443/v1/models**
- **:443/v1/chat/completions**
- **:443/v1/completions**
- **:443/v1/embeddings**
- **:443/tokenize**
- **:443/detokenize**



NOTE

- The vLLM runtime is compatible with the OpenAI REST API.
- To use the embeddings inference endpoint in vLLM, you must use an embeddings model that the vLLM supports. You cannot use the embeddings endpoint with generative models. For more information, see [Supported embeddings models in vLLM](#).
- As of vLLM v0.5.5, you must provide a chat template while querying a model using the **/v1/chat/completions** endpoint. If your model does not include a predefined chat template, you can use the **chat-template** command-line parameter to specify a chat template in your custom vLLM runtime, as shown in the example. Replace **<CHAT_TEMPLATE>** with the path to your template.

```
containers:
  - args:
    - --chat-template=<CHAT_TEMPLATE>
```

You can use the chat templates that are available as **.jinja** files [here](#) or with the vLLM image under **/app/data/template**. For more information, see [Chat templates](#).

As indicated by the paths shown, the model serving platform uses the HTTPS port of your OpenShift router (usually port 443) to serve external API requests.

Example command

```
curl -v https://<inference_endpoint_url>:443/v1/chat/completions -H "Content-Type: application/json" -
```

```
d '{ "messages": [{ "role": "<role>", "content": "<content>" }] }' -H 'Authorization: Bearer <token>'
```

4.4.4. vLLM Intel Gaudi Accelerator ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

4.4.5. vLLM AMD GPU ServingRuntime for KServe

See [vLLM NVIDIA GPU ServingRuntime for KServe](#).

4.4.6. vLLM Spyre AI Accelerator ServingRuntime for KServe



IMPORTANT

Support for IBM Spyre AI Accelerators on x86 is currently available in Red Hat OpenShift AI 3.0 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

For more information about the support scope of Red Hat Technology Preview features, see [Technology Preview Features Support Scope](#).

You can serve models with IBM Spyre AI accelerators on x86 by using the **vLLM Spyre AI Accelerator ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

4.4.7. vLLM Spyre s390x ServingRuntime for KServe

You can serve models with IBM Spyre AI accelerators on IBM Z (s390x architecture) by using the **vLLM Spyre s390x ServingRuntime for KServe** runtime. To use the runtime, you must install the Spyre Operator and configure a hardware profile. For more information, see [Spyre operator image](#) and [Working with hardware profiles](#).

4.4.8. NVIDIA Triton Inference Server

REST endpoints

- **v2/models[/versions/<model_version>]/infer**
- **v2/models/<model_name>[/versions/<model_version>]**
- **v2/health/ready**
- **v2/health/live**
- **v2/models/<model_name>[/versions/]/ready**
- **v2**

Example command


```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data> ]}]}' -H 'Authorization: Bearer <token>'
```

gRPC endpoints

- **:443 inference.GRPCInferenceService/ModelInfer**
- **:443 inference.GRPCInferenceService/ModelReady**
- **:443 inference.GRPCInferenceService/ModelMetadata**
- **:443 inference.GRPCInferenceService/ServerReady**
- **:443 inference.GRPCInferenceService/ServerLive**
- **:443 inference.GRPCInferenceService/ServerMetadata**

Example command

```
grpcurl -cacert ./openshift_ca_istio_knative.crt -proto ./grpc_predict_v2.proto -d @ -H "Authorization:  
Bearer <token>" <inference_endpoint_url>:443 inference.GRPCInferenceService/ModelMetadata
```

4.4.9. Seldon MLServer

REST endpoints

- **v2/models/[versions/<model_version>]/infer**
- **v2/models/<model_name>[versions/<model_version>]**
- **v2/health/ready**
- **v2/health/live**
- **v2/models/<model_name>[versions]/ready**
- **v2**

Example command

```
curl -ks <inference_endpoint_url>/v2/models/<model_name>/infer -d '{ "model_name": "  
<model_name>", "inputs": [{ "name": "<name_of_model_input>", "shape": [<shape>], "datatype": "  
<data_type>", "data": [<data> ]}]}' -H 'Authorization: Bearer <token>'
```

gRPC endpoints

- **:443 inference.GRPCInferenceService/ModelInfer**
- **:443 inference.GRPCInferenceService/ModelReady**
- **:443 inference.GRPCInferenceService/ModelMetadata**

- **:443 inference.GRPCInferenceService/ServerReady**
- **:443 inference.GRPCInferenceService/ServerLive**
- **:443 inference.GRPCInferenceService/ServerMetadata**

Example command

```
grpcurl -cacert ./openshift_ca_istio_knative.crt -proto ./grpc_predict_v2.proto -d @ -H "Authorization: Bearer <token>" <inference_endpoint_url>:443 inference.GRPCInferenceService/ModelMetadata
```

4.4.10. Additional resources

- [Caikit API documentation](#)
- [OpenVINO KServe-compatible REST API documentation](#)
- [OpenAI API documentation](#)
- [Open Inference Protocol](#)
- [Supported model-serving runtimes.](#)